

1. Présentation du projet

Dataset, problème métier, variable cible et contraintes

Ce projet construit une chaîne MLOps complète, de la donnée brute jusqu'à une API de prédiction supervisée, pour anticiper le retard d'un vol à l'arrivée. Il a été réalisé en équipe de trois (Hamid, Joan et moi) en une semaine ; le sujet mentionne un travail « individuel ou en binôme » (à clarifier, nous étions trois). Le code est publié sur un dépôt GitHub : github.com/Justhamid/FlightDelaysMLOps_evaluation2026.

Problème métier. Une compagnie aérienne veut anticiper les retards pour optimiser ses opérations au sol et informer les passagers en avance. La prédiction doit donc être disponible **avant le départ** : un modèle exploitable seulement une fois le vol parti n'aurait aucune valeur.

Dataset. Jeu Kaggle *usdot/flight-delays* (`flights.csv`, ~5,8 millions de vols, 565 Mo). Un échantillon de 200 000 vols est tiré aléatoirement avec une graine fixe (reproductibilité) et versionné avec DVC.

Variable cible. Binaire : `delayed` = 1 si `ARRIVAL_DELAY` > 15 min, sinon 0. Le taux de retard est d'environ **17,8 %** — un jeu nettement déséquilibré, ce qui aura un impact direct sur le modèle.

Variables d'entrée. Uniquement celles connues avant le départ : mois, jour de la semaine, compagnie, aéroports de départ et d'arrivée, heure de départ programmée, durée de vol programmée, distance. Les colonnes connues seulement après le vol (retard au départ, temps de roulage...) sont volontairement exclues pour éviter une **fuite de données** (*data leakage*) qui rendrait le modèle artificiellement parfait mais inutilisable en réel.

Contraintes identifiées

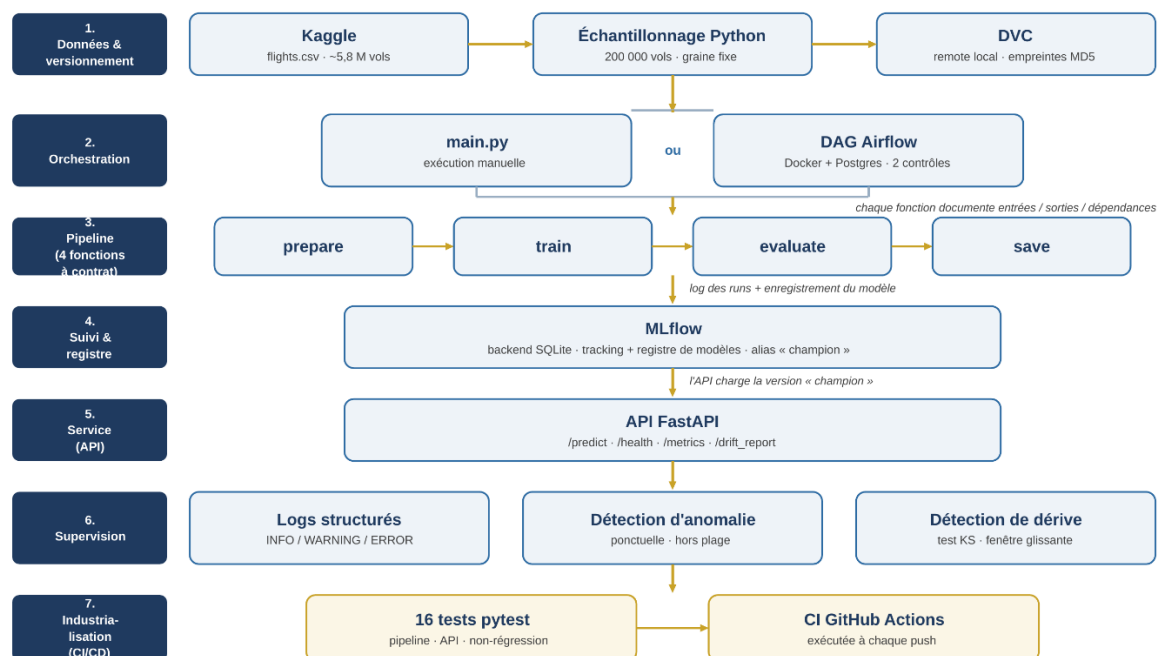
- **Volume** : dataset trop lourd pour un entraînement direct — échantillonnage nécessaire.
- **Délai** : une semaine pour couvrir toute la chaîne MLOps.
- **Environnement** : postes Windows — Docker requis pour Airflow, non natif sous Windows.
- **Budget** : aucun budget cloud, tout tourne en local.

2. Architecture du système

Enchaînement des briques, de la donnée brute à l'API supervisée

Le flux est continu. Les données Kaggle (flights.csv) sont réduites par un **script Python d'échantillonnage**, puis l'échantillon est **versionné avec DVC** (remote local). Le cœur est un **pipeline en 4 fonctions à contrat** (prepare → train → evaluate → save, dans src/pipeline.py), lançable **manuellement** (main.py) ou **orchestré** par un **DAG Airflow** (Docker, Postgres) avec 2 contrôles. Le suivi et le registre de modèles sont assurés par **MLflow** (SQLite, alias « champion »). Une **API FastAPI** charge le modèle via cet alias et expose /predict, /health, /metrics, /drift_report. La **supervision** couvre logs structurés, anomalie ponctuelle et dérive (test KS sur fenêtre glissante). Enfin, **16 tests pytest** sont joués par une **CI GitHub Actions** à chaque push.

Architecture du système — pipeline MLOps de prédiction des retards de vols



Légende suggérée : Kaggle (flights.csv) → échantillonnage Python → données versionnées (DVC, remote local) → pipeline 4 fonctions (prepare / train / evaluate / save) → orchestration manuelle (main.py) ou DAG Airflow (Docker + Postgres, 2 contrôles) → tracking & registre MLflow (SQLite, alias « champion ») → API FastAPI (/predict, /health, /metrics, /drift_report) → supervision (logs, anomalie, dérive KS) → tests pytest + CI GitHub Actions. Flèches pleines = flux de données ; regrouper visuellement les couches « orchestration », « suivi des modèles » et « industrialisation (CI/CD) ».

3. Choix techniques

Choix structurants, compromis assumés et sécurité

3.1 Données / DVC. Échantillon de 200 000 lignes (pas le dataset complet) pour des temps d'entraînement raisonnables ; DVC en **remote local** (`../dvcstore`), sans cloud. *Compromis* : reproductible en local mais chaque membre doit configurer le même remote — pas un vrai partage distant.

3.2 Pipeline / MLflow. Pipeline en 4 fonctions à contrat documentées (entrées / sorties / dépendances). MLflow avec backend **SQLite** (pas de serveur permanent), Model Registry natif ; 4 runs RandomForest comparés. Point notable : sans `class_weight="balanced"`, le déséquilibre (~18 %/82 %) faisait prédire toujours « pas de retard » ($F1 = 0$) ; ce paramètre a permis un **F1 exploitable (~0,36)**, là où la profondeur seule ne suffisait pas.

3.3 Orchestration Airflow. DAG dans Docker (Postgres + scheduler + webserver, **LocalExecutor** — ni Celery ni Redis pour ce volume), réutilisant directement les fonctions du pipeline (pas de duplication). 2 contrôles : format des données en entrée, artefact modèle en sortie. Testé en réel : un échec volontaire (fichier renommé) stoppe le DAG sans modèle corrompu (tâches suivantes en `upstream_failed`).

3.4 API FastAPI. 3 routes (`/`, `/health`, `/predict`) + Swagger. Deux **validateurs métier** au-delà du typage : heure HHMM valide (minutes < 60) et aéroport de départ $\neq$ aéroport d'arrivée. Gestion d'erreurs explicite (422 validation, 500 interne).

3.5 Supervision. Logs structurés (INFO/WARNING/ERROR) et route `/metrics` (6 indicateurs). Deux mécanismes distincts : la **détection d'anomalie** ponctuelle (valeur hors plage d'entraînement) bloque une seule prédiction sans erreur HTTP ; la **dérive** (test de Kolmogorov-Smirnov sur les 200 dernières requêtes vs échantillon d'entraînement) alerte sur un déplacement global de la distribution sans jamais bloquer.

3.6 Tests / CI. 16 tests pytest (5 pipeline, 7 API, 3 non-régression) s'appuyant sur un échantillon fixe (500 lignes committées) avec un modèle de test injecté, sans MLflow / DVC / Docker — CI rapide et fiable. *Compromis* : ce n'est pas un test d'intégration bout-en-bout du vrai pipeline.

Sécurité. Secrets : aucun secret sensible (seul `.env` = config Airflow non sensible), pas de gestionnaire type Vault — à prévoir. **Validation des entrées** : Pydantic strict + 2 validateurs métier, erreurs 422 avant le modèle. **Droits d'accès** (limite assumée) : API sans authentification, Airflow en `admin/admin`, MLflow UI sans contrôle — à corriger avant prod (token API, identifiants changés, réseau restreint). **Intégrité** : DVC garantit le dataset (MD5), MLflow trace la provenance des modèles ; pas de checksum au chargement par l'API. **Limites actuelles** : HTTP local non chiffré, pas de scan de dépendances, remote DVC local non sécurisé.

4. Captures d'écran obligatoires

MLflow (runs comparés), pipeline orchestré (succès / échec), CI verte

4.1 — MLflow UI (runs comparés)

Experiments > flight_delays >

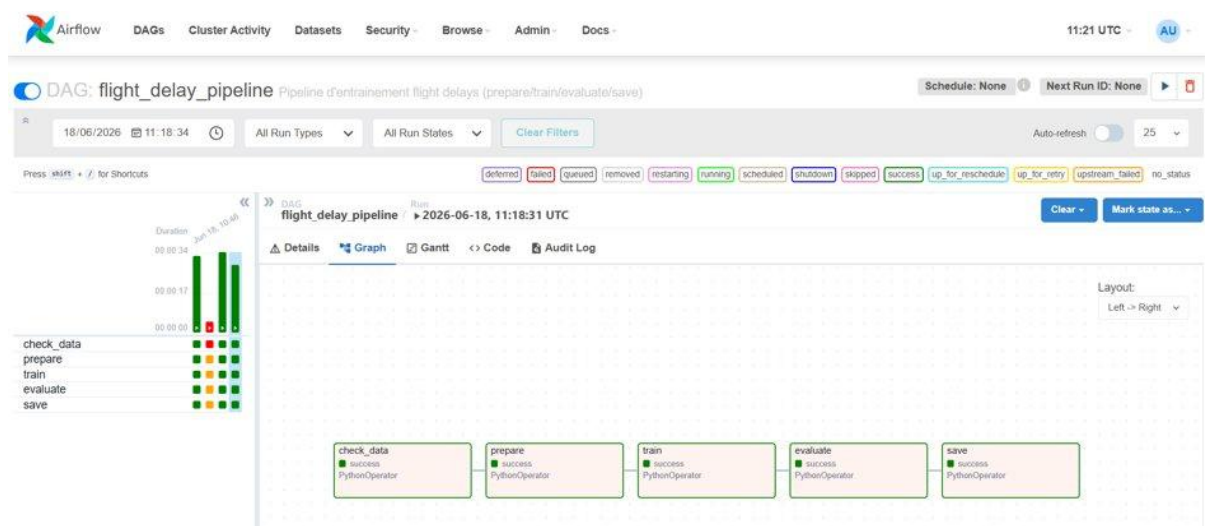
Runs

Q metrics.rmse < 1 and params.model = "tree" All time State: Active Datasets Sort: Created Columns + New run

Group by

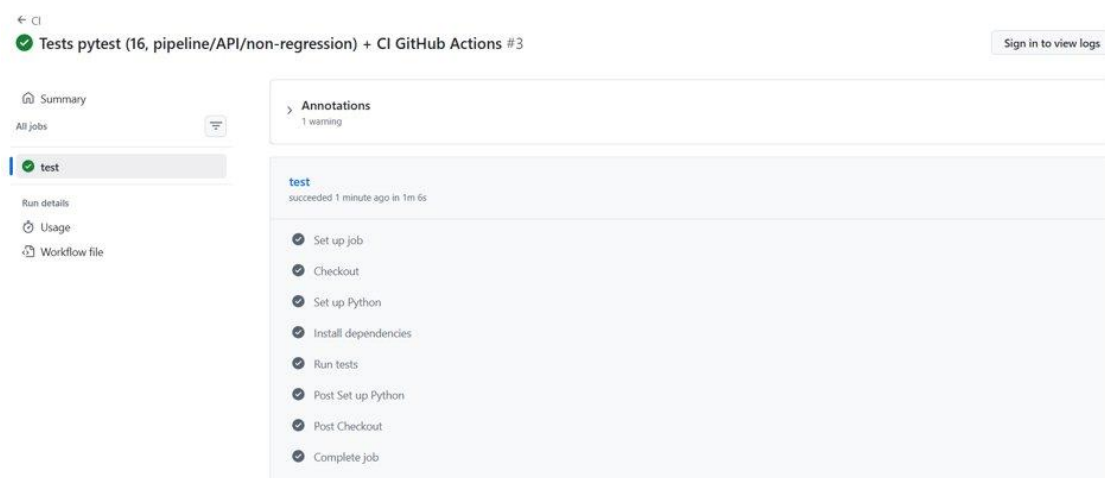
	Run Name	Created	Duration	Models	accuracy	f1	precision	recall	class_weight	max_depth	n_estimators
	marvelous-smelt-15	1 hour ago	20.1s	flight_delay_model v4	0.624	0.359618496...	0.258176927...	0.592369196...	balanced	20	300
	carefree-bear-445	1 hour ago	10.9s	flight_delay_model v3	0.60095	0.355955455...	0.249844236...	0.618740356...	balanced	15	150
	gifted-gull-485	1 hour ago	10.5s	flight_delay_model v2	0.821775	0	0	0	-	15	150
	salty-sloth-7	1 hour ago	6.7s	flight_delay_model v1	0.821775	0	0	0	-	5	50

4.2 — Pipeline orchestré sous Airflow (un run réussi, un échoué)



Vue Graph : *check_data* → *prepare* → *train* → *evaluate* → *save*, 5 tâches en succès. À gauche, la vue Grid montre l'isolation des échecs — sur le run échoué, *check_data* passe en rouge et les tâches suivantes en *upstream_failed* ne sont jamais exécutées.

4.3 — Intégration continue verte (GitHub Actions)



CI GitHub Actions : les 16 tests pytest passent au vert, déclenchés automatiquement à chaque push.

5. Bilan

Ce qui a marché, les difficultés, les améliorations et le passage en production

Ce qui a marché. L'architecture complète a été construite et testée brique par brique en une semaine. Tous les indicateurs obligatoires sont au vert : suivi MLflow, orchestration Airflow, API de prédiction, supervision (métriques, anomalie, dérive) et CI verte à chaque push.

Ce qui a posé problème

- **Déséquilibre de classes** : modèle inutile ($F1 = 0$) prédisant toujours « pas de retard » tant que `class_weight` n'était pas réglé.
- **Bug API réel** : les erreurs des validateurs métier faisaient basculer le 422 en 500, un `ValueError` brut n'étant pas sérialisable en JSON — corrigé avec `jsonable_encoder`.
- **Processus fantômes** : d'anciens serveurs oubliés (UI MLflow, uvicorn) répondaient encore sur les mêmes ports, donnant des résultats trompeurs — diagnostiqué via `netstat`.
- **Conflit de versions** : un coéquipier a poussé en parallèle une implémentation complète sur le même dépôt ; conflit résolu en choisissant une version de référence.

Ce que nous ferions différemment

- Mettre en place un **vrai remote DVC partagé** (cloud) dès le début plutôt qu'un remote local.
- Définir une **répartition claire du travail** avant de coder en parallèle, pour éviter les doublons.

Ce qui resterait à faire pour un vrai déploiement

- **Authentification** sur l'API et sur Airflow, et passage en **HTTPS**.
- **Gestion des secrets** via un gestionnaire dédié.
- **Évolution du stockage** pour absorber des volumes bien plus importants et conserver un **historique réel des prédictions** plutôt qu'un suivi en mémoire.